Queueing the Transformer

LLM inference as an operations research problem:
a validated discrete-event model of batched serving

Shaan Jain
github.com/Shaandjain/llm-inference-queueing

June 2026

Abstract

LLM serving systems are queueing systems: requests arrive stochastically, occupy a batched server through distinct prefill and decode phases, and contend for KV-cache memory. I built a discrete-event simulator of batched LLM inference on a deliberately small cost model — two linear coefficients per phase — and asked whether it can *predict* a real serving system rather than merely illustrate one. Fitting the model from 40 sequential probe requests and a concurrency sweep against vLLM running Qwen2.5-7B-Instruct on an NVIDIA L4, then replaying held-out Poisson workloads open-loop, the simulator predicted median time-to-first-token within 12% and, after correcting a systematic error, end-to-end latency percentiles within 6–13% at both 50% and 80% of saturation load. The correction itself is the most transferable finding: a latency intercept fitted over a network conflates per-request overhead with per-iteration GPU time, and charging it as GPU-blocking time inflates predicted tail latency by 37–60%. Along the way the simulation reproduces the canonical continuous-vs-static batching results and surfaces an under-stated operational fact: busy-time "utilization" of a batched LLM server saturates near 100% at *every* load level, making GPU-busy metrics useless as load signals; decode batch occupancy is the signal that tracks load.

1 Introduction

When an AI agent feels slow, the bottleneck is usually not the model's intelligence but the serving system's queueing behavior: how requests are batched, when prefill preempts decode, and how much headroom the operator left below saturation. These are classical operations research questions — arrival processes, service-time distributions, scheduling policies, tail latency — wearing a GPU costume.

This report does three things:

1. **Models** batched LLM serving as a discrete-event system with a two-phase linear cost model (§2), implementing both iteration-level (continuous) batching in the style of Orca [1] and vLLM [2], and classic request-level (static) batching.

2. **Reproduces** the canonical serving results from first principles (§3): a $\sim 5\times$ capacity gap between continuous and static batching, the tail-latency hockey stick, and the cost of arrival burstiness — plus one less commonly plotted fact about utilization metrics.

3. **Validates** the simulator against a real deployment (§4): vLLM 0.11 serving Qwen2.5-7B-Instruct on an NVIDIA L4, with coefficients fitted from cheap probes and accuracy assessed on held-out workloads the fitting never saw. Total GPU cost of the validation protocol: about $1.

Nothing in §3 is a new result; simulators of LLM serving exist, notably Vidur [3]. The contribution here is methodological and pedagogical: a compact (under 1,000 lines), fully open simulator with a fit-predict-attribute validation loop, and two documented measurement traps (§5) that anyone benchmarking inference over a network will encounter.

2 Model

Requests. A request r arrives at time a_r with p_r prompt tokens and g_r tokens to generate. Workload profiles (`chat`, `rag`, `agent`) draw (p_r, g_r) from lognormal distributions; arrivals are Poisson or an on/off bursty process calibrated to the same mean rate.

Server. The engine executes iterations. A *prefill* iteration processes the prompts of newly admitted requests and emits their first token; a *decode* iteration generates one token for every active sequence. Iteration costs are linear:

$$T_{\text{prefill}}(n_{\text{tok}}) = \beta_p + \kappa_p\, n_{\text{tok}}, \qquad T_{\text{decode}}(b) = \beta_d + \kappa_d\, b, \tag{1}$$

where n_{tok} is total prompt tokens in the iteration and b is the decode batch size. KV-cache occupancy is bounded by a capacity parameter; admission reserves a request's full footprint $p_r + g_r$.

Schedulers. *Continuous batching* admits requests between iterations up to a batch-size cap and prefill token budget, prioritizing prefill when admissible work is waiting (vLLM's default disposition). *Static batching* takes up to B requests, prefills them together, and decodes until the longest request finishes; completed sequences pad the batch at full cost.

Simplifications. The scheduler knows g_r at admission (no preemption is modeled); prefill is never chunked or overlapped with decode; the decode cost ignores KV-length effects ($\kappa_{kv} = 0$). §4 quantifies what these simplifications cost.

3 Simulation findings

All experiments use 1,500-request workloads, 5 seeds, and min–max bands; load is expressed as a fraction of each policy's *measured* saturation throughput so policies are compared fairly. One command regenerates every figure from raw traces.

3.1 Continuous vs. static batching

On the chat profile, continuous batching saturates at 5.10 requests/s against 1.05 for static batch-8 — the expected ∼5× gap [1, 2], here derived from queueing structure alone (Figure 1). Raising the static batch to 32 lifts capacity only to 1.70 rps, still below continuous batching capped at batch 8 (2.18 rps): padding waste does not average out (Figure 2).

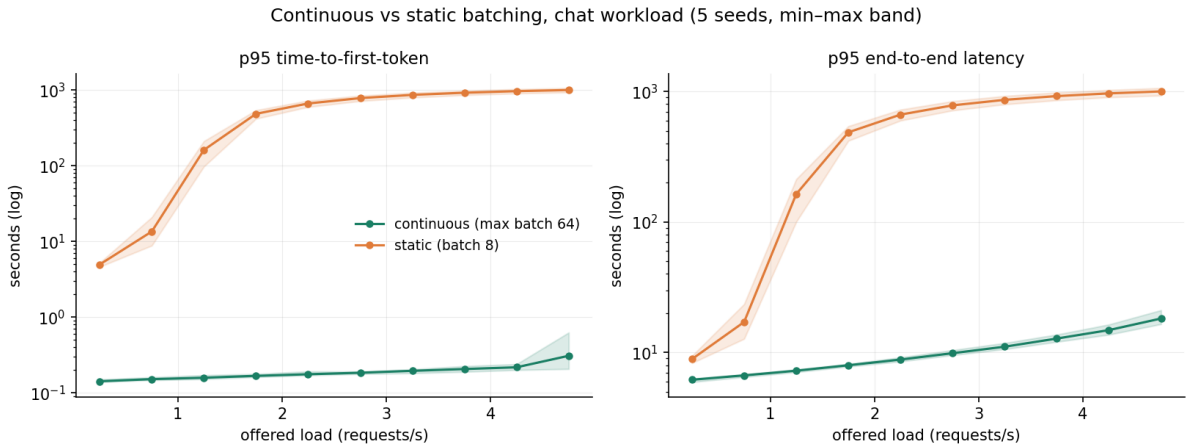

Figure 1: p95 TTFT and end-to-end latency vs. offered load, chat workload. Static batching's tail explodes past ∼1 rps while continuous batching serves 4.75 rps at sub-second p95 TTFT.

3.2 "Busy" is not a load signal

Plotting p95 latency against busy-time utilization produced a degenerate figure: utilization is ≥ 0.97 at *every* offered load from 30% to 98% of capacity. The server never idles — at low load it simply runs underfilled batches, paying nearly the full per-iteration cost (β_d dominates) to produce few tokens. The metric that actually tracks load is decode batch occupancy (Figure 3). Operationally: a dashboard showing "GPU busy: 99%" says nothing about headroom; mean batch occupancy and queue depth do.

3.3 Burstiness

At identical mean arrival rates, an on/off arrival process (4× intensity bursts, 20% duty cycle) roughly doubles p95 TTFT relative to Poisson across all load levels on the agent profile (Figure 4). Capacity planning on mean load alone systematically under-provisions for agentic traffic, which is naturally bursty (tool-call fan-outs, retries).

4 Validation against vLLM on an L4

A simulator that only draws plausible curves is decoration. The test: fit the cost model from cheap probes, then predict latency distributions for workloads the fitting never saw.

4.1 Protocol

Serve. vLLM 0.11.0, Qwen2.5-7B-Instruct (bf16), NVIDIA L4 on Modal; `max_num_seqs` 64, `max_model_len` 8192. The client runs on a laptop in Toronto; all measurements traverse the public internet, which turns out to matter (§4.2).

Fit. (i) A sequential prompt-length sweep (40 requests, 63–2,873 prompt tokens) fits the prefill line: $T_{\text{TTFT}} = 271.6\,\text{ms} + 0.288\,\text{ms/token}$ ($R^2 = 0.887$). (ii) A concurrency sweep ($c \in \{1, 2, 4, 8, 16, 32\}$, fixed shapes, forced 128-token outputs) fits decode: $T_{\text{TPOT}} = 56.9\,\text{ms} +$

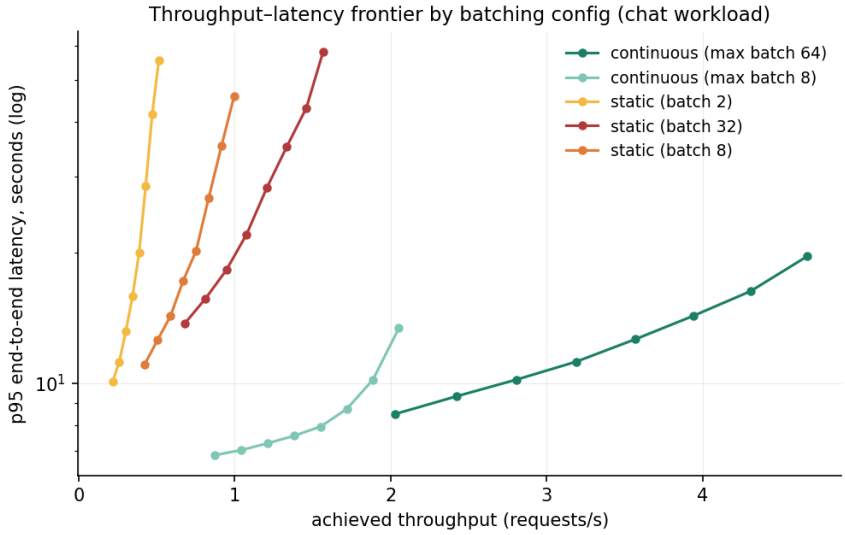

Figure 2: Throughput–latency frontiers by batching configuration. Every static configuration is dominated by continuous batching at any batch cap.

0.67 ms/seq ($R^2 = 0.702$). Batching is nearly free up to c=16 (58.7 ms → 66 ms per token) — the measured economics behind §3.

Predict and replay. The fitted simulator predicts L4 saturation at 1.46 rps on the chat profile. Two held-out Poisson workloads (250 requests each, unseen seeds) were replayed open-loop at 0.73 and 1.17 rps — 50% and 80% of predicted capacity — with exact output lengths forced via `ignore_eos` so simulator and server process identical request sets. Maximum dispatch lag was 18 ms, so the intended arrival process survived contact with the event loop.

4.2 Results and error attribution

Model variant	Load	p50 TTFT	p95 TTFT	p50 e2e	p95 e2e
raw fit	0.73 rps	−12%	−7%	+37%	+41%
raw fit	1.17 rps	−10%	−3%	+60%	+51%
overhead-attributed	0.73 rps	−15%	−33%	**+6%**	**+9%**
overhead-attributed	1.17 rps	−15%	−33%	**+13%**	**+11%**

Table 1: Prediction error (predicted vs. observed percentiles, n=250 per run). Observed values at 1.17 rps: p50/p95 TTFT 0.45/0.72 s; p50/p95 e2e 9.6/35.2 s.

The raw fitted model nails TTFT but overpredicts end-to-end latency by 37–60% (Table 1). The cause is an attribution error, not a noise problem. The 272 ms prefill intercept is dominated by *per-request* overhead — network round trips, API processing, tokenization, scheduling delay — but the simulator charges β_p on every prefill *iteration* as GPU-blocking time during which all decodes stall. Real vLLM overlaps chunked prefill with decode, and a network round trip blocks nobody's decode. At 0.73 rps over a 354 s window, the phantom serialized time is roughly $258 \times 240\,\text{ms} \approx 62\,\text{s}$ of fictitious server busyness.

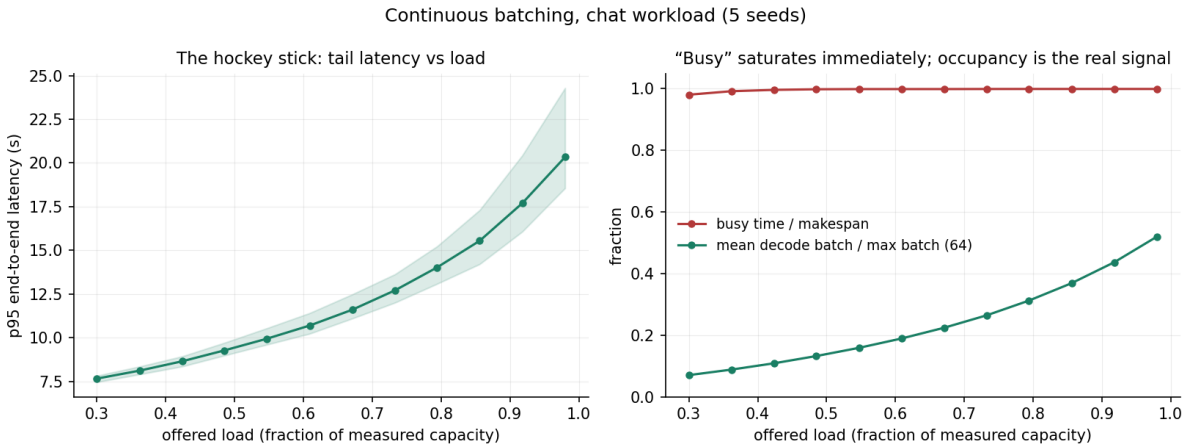

Figure 3: Left: the tail-latency hockey stick vs. load. Right: busy-time utilization is flat at ≈1.0 across the entire sweep, while batch occupancy rises with load — occupancy, not busyness, is the real signal.

Re-running the prediction with 240 ms of the intercept reattributed as non-blocking per-request overhead collapses e2e error to +6–13% at both load levels (Figure 5); the predicted and observed e2e CDFs nearly coincide. The correction is not free: a constant offset cannot model TTFT *variance*, so predicted TTFT tails become too tight (−33% at p95). A proper treatment would fit per-request overhead as a distribution and model chunked prefill; both are future work.

5 Measurement traps

Two failure modes consumed real debugging time and generalize to any inference benchmarking effort.

Prefix caching invalidates repeated prompts. An earlier fitting run against a local Ollama server (qwen2.5:3b, Apple M3 Pro) produced repetitions with flat 0.13 s TTFT regardless of prompt length, against 5.6 s cold at 2,834 tokens: the server's prompt cache skipped prefill for identical prompts, so three of four "repetitions" measured the cache. Servers cache by longest matching prefix; the fix is a unique nonce at the *start* of every benchmark prompt. (With caching defeated, the same protocol fit the M3 Pro cleanly: $10.2\,\text{ms} + 1.925\,\text{ms/token}$, $R^2 = 0.999$.)

Fitted intercepts conflate overhead classes. As §4.2 shows, a latency intercept fitted over a network bundles per-request costs with per-iteration GPU costs. Any model that serializes the former across concurrent work will overpredict latency under load — by 37–60% in this setup. Separating the classes requires either server-side timing or an explicit overhead term; assuming the split is the current method's main internal threat to validity.

6 Limitations

The validation covers one model, one GPU, one workload family, two below-saturation load levels, and a single 250-request replay per level. Chunked prefill is not modeled; the 240 ms

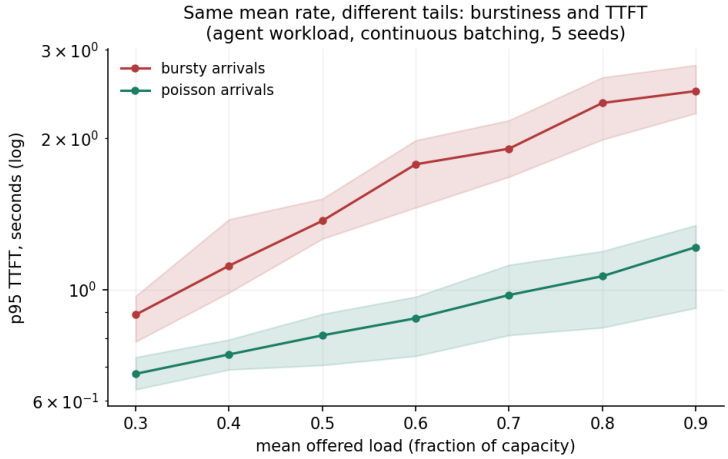

Figure 4: Same mean rate, different tails: bursty arrivals vs. Poisson, agent workload.

overhead split is an ablation estimate rather than an independent measurement; predicted TTFT carries no variance model; KV-length effects on decode cost are disabled; and the simulator's admission control reserves full output length, which real schedulers cannot know. The simulation experiments in §3 use hand-set placeholder coefficients, so their *comparisons* are the claims, not their magnitudes.

7 Outlook

The natural next step is to use the queueing frame for what it is good at: deriving operating rules. Heavy-traffic approximations should yield closed-form headroom rules of the form "for this arrival burstiness and service-time CV, run at ρ^* to hold p95 below X" — checkable against the L4 traces already collected. On the systems side: chunked-prefill modeling, fitted overhead distributions, and scheduler variants (shortest-prefill-first, deadline-aware) that the simulator can evaluate without renting another GPU.

A Reproduction

```
git clone https://github.com/Shaandjain/llm-inference-queueing
uv sync && uv run pytest                        # 10 tests
uv run python experiments/run_experiments.py    # 350 sims (~1 min, CPU)
uv run python analysis/make_plots.py            # figures 1-4
# validation (requires a Modal account; ~$1 of L4 time):
modal deploy serving/vllm_modal.py
uv run python live/capture_traces.py    --base-url <URL>/v1 ...
uv run python live/concurrency_sweep.py --base-url <URL>/v1 ...
uv run python live/fit_coefficients.py  results/l4-prefill \
    --concurrency-dir results/l4-conc --label modal-l4-vllm-qwen2.5-7b
uv run python live/replay_workload.py   --rate 0.73 --seed 42 ...
uv run python analysis/validate.py --request-overhead-s 0.24 ...
modal app stop -y vllm-l4-qwen7b
```

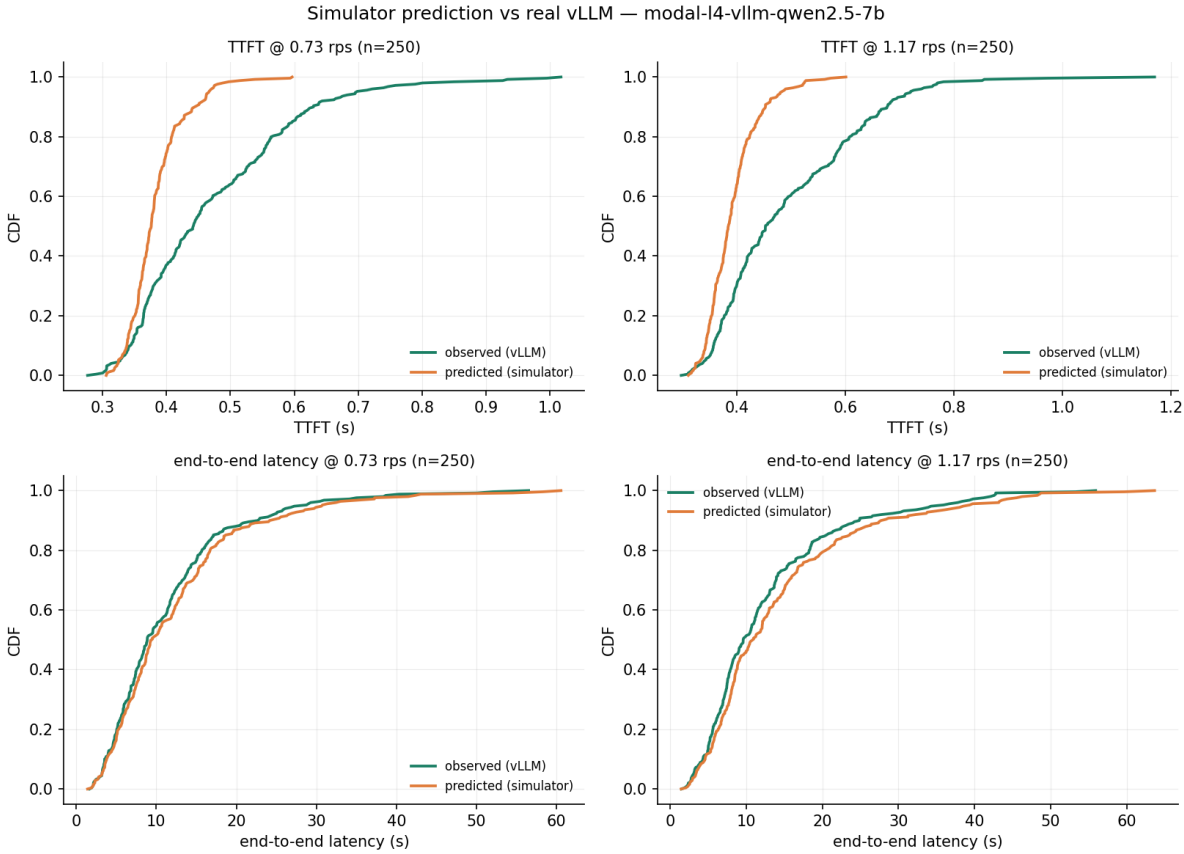

Figure 5: Predicted vs. observed CDFs after overhead attribution, both replay loads. Bottom row: end-to-end latency distributions nearly coincide. Top row: predicted TTFT is correct in location but under-dispersed.

All raw traces ship in the repository in a shared JSONL schema; every figure and table regenerates from them.

B A note on version skew

The first deployment failed at server start: vLLM 0.11.0 resolved against transformers 5.x, which removed `all_special_tokens_extended`. The fix is pinning `transformers==4.57.1` alongside the vLLM pin. Serving images should pin the full inference stack, not just the engine.

References

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. *OSDI*, 2022.

[2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023. arXiv:2309.06180.

[3] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. Gulavani, R. Ramjee, and A. Tumanov. Vidur: A large-scale simulation framework for LLM inference. *MLSys*, 2024.

[4] L. Zheng, L. Yin, Z. Xie, et al. SGLang: Efficient execution of structured language model programs. *NeurIPS*, 2024.